

DEEP LEARNING

Unit – II

Convolutional Networks

Rajatha

Assistant Professor

Information Science and Engineering

RV College of Engineering

Convolutional Networks

- **Convolutional networks** known as **Convolutional Neural Networks(CNN)**
- Specialized kind of neural network for processing data that has a known, **grid-like topology**
- “Convolutional neural network” employs a mathematical operation called **convolution**
- Convolution is a **specialized kind of linear operation**
- *Convolutional networks are simply neural networks that **use convolution in place of general matrix multiplication** in at least one of their layers*

Convolution Operation

- Convolution is an operation on two functions of a real valued argument

- Example

- ☐ Assume tracking the location of a spaceship with a laser sensor
- ☐ Laser sensor provides an **output $x(t)$** , position of the spaceship **at time t**
- ☐ Both x and t are real-valued

(we get a different reading from the laser sensor at any instant in time)

- Suppose laser sensor is noisy?

Noisy Laser Sensor

- To obtain a less noisy estimate of the spaceship's position, **we average** together several measurements
- Recent measurements are more relevant, hence **weighted average** is taken
- Weighting function $w(a)$, where **a is the age of a measurement**
- Weighted average operation results in a new function “ s ” -a smoothed estimate of the position of the spaceship

$$s(t) = \int x(a)w(t - a)da$$

- This operation is called **convolution**.

Convolution Operation

- The convolution operation is typically denoted with an asterisk

$$s(t) = (x * w)(t)$$

- “ w ” needs to be 0 for all negative arguments

- In convolutional

- ☐ The first argument (in this example, the function x) to the convolution is often referred to as the **input**
- ☐ Second argument (in this example, the function w) as the **kernel**.
- ☐ The output is sometimes referred to as the **feature map**.

Laser Sensor

- Providing measurements at every instant in time is **not realistic**
- Time will **be discretized**, and sensor will provide data at **regular intervals**
- Assume laser provides a measurement **once per second**
- The time index t can then take on only **integer values**
- If we now assume that x and w are defined only on integer t , we can define the discrete convolution:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a)$$

Tensors

- In machine learning applications
 - Input is a multidimensional array of data
 - kernel is a multidimensional array of parameters
- These multidimensional arrays are **tensors**

Convolution is commutative

- if we use a two-dimensional image I as our input, we probably also want to use a two-dimensional kernel K :

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$$

Convolution is commutative, meaning we can equivalently write:

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

Usually the latter formula is more straightforward to implement in a machine learning library, because there is less variation in the range of valid values of m and n .

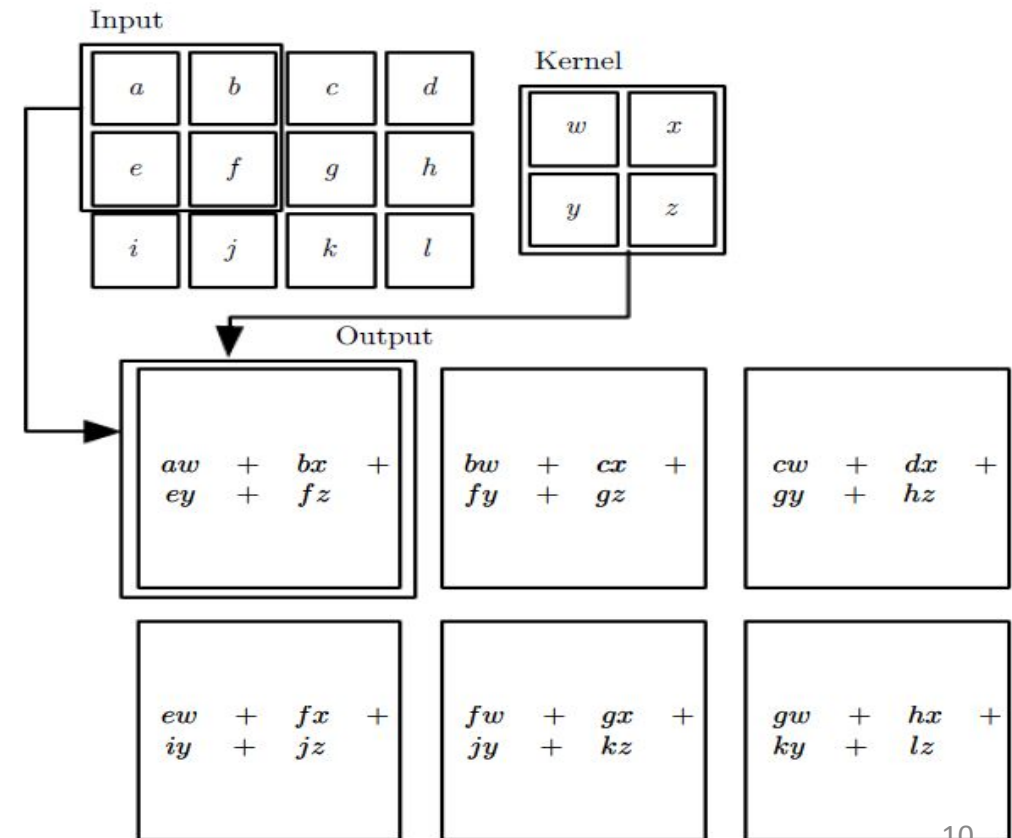
Cross-correlation

- Instead, many neural network libraries implement a related function called the **cross-correlation**, which is the same as convolution but without flipping the kernel

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

Convolution applied to a 2-D tensor

- example of convolution (without kernel flipping) applied to a 2-D tensor.



Motivation

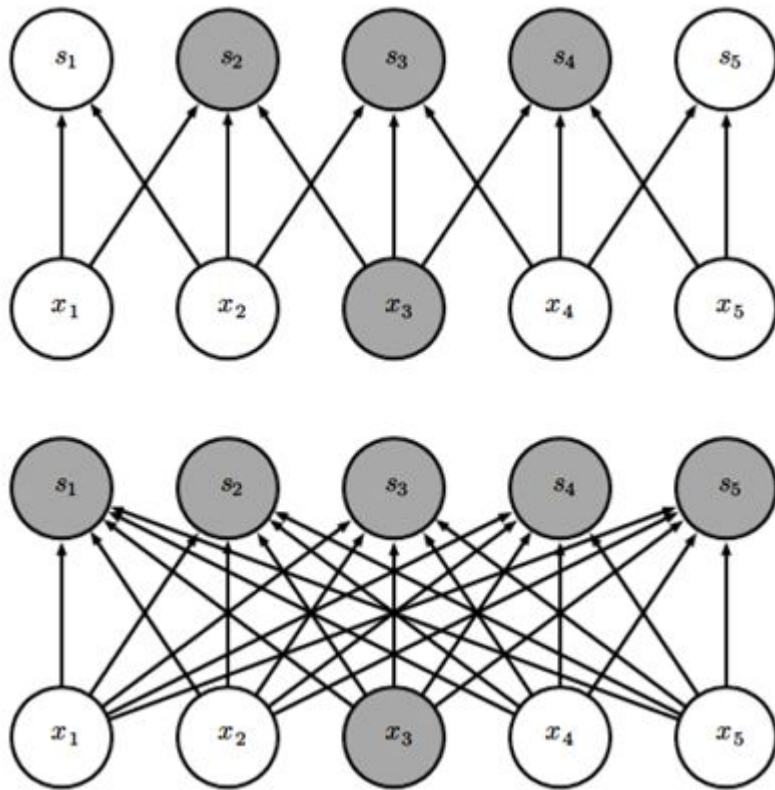
- Convolution leverages three important ideas that can help improve a machine learning system
 - ❑ **Sparse interactions**
 - ❑ **Parameter sharing**
 - ❑ **Equivariant representations.**

Convolution provides a means for working with inputs of variable size

Sparse interactions

- Convolutional networks, however, typically have **sparse interactions** (also referred to as **sparse connectivity** or **sparse weights**).
- Accomplished by **making kernel smaller** than the input
- Processing an image, have thousands of pixels, but we can detect small meaningful features such as edges with kernels
- Store **fewer parameters, reduces memory** and improves statistical efficiency
- Computing the **output requires fewer** operations
- If there are m inputs and n outputs, then **matrix multiplication** requires $m \times n$ parameters and the algorithms used in **practice have $O(m \times n)$ runtime**
- If we limit the number of connections **sparingly connected** approach requires only $k \times n$ parameters and **$O(k \times n)$ runtime**
- Possible to obtain good performance on the machine learning task while **keeping k several orders of magnitude smaller than m** .

Sparse interactions

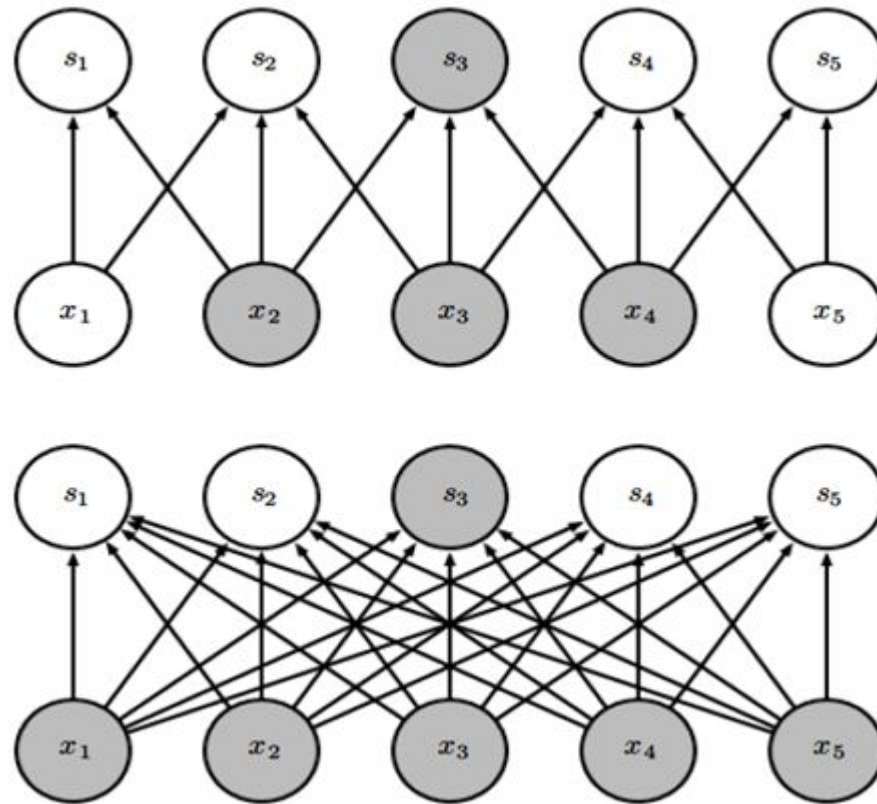


(Top) When s is formed by convolution with a kernel of width 3, only three outputs are affected by x .

(Bottom) When s is formed by matrix multiplication, connectivity is no longer sparse, so all of the outputs are affected by x_3 .

Figure 9.2: Sparse connectivity, viewed from below:

Sparse interactions

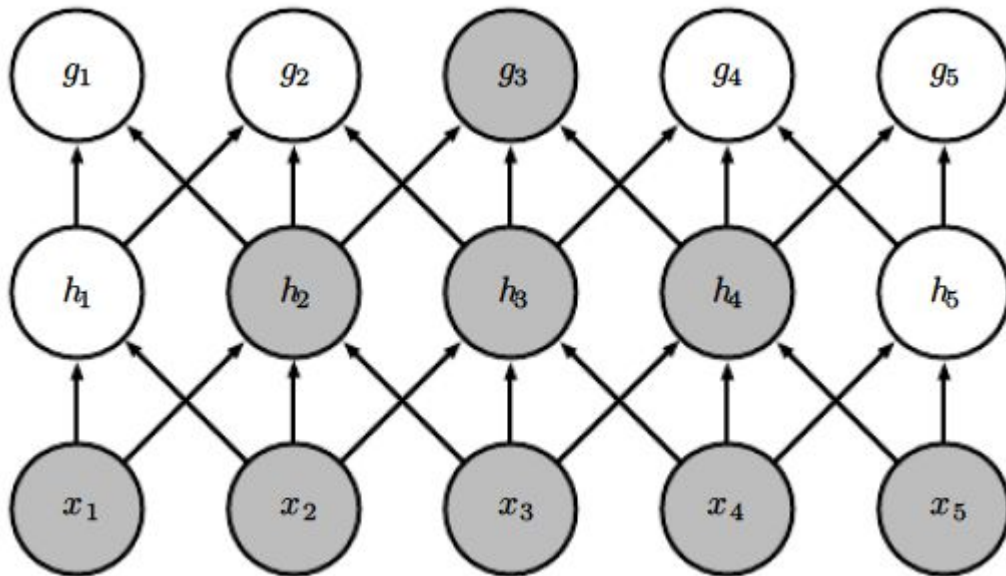


(Top) When s is formed by convolution with a kernel of width 3, only three inputs affect s_3 .

(Bottom) When s is formed by matrix multiplication, connectivity is no longer sparse, so all of the inputs affect s_3 .

Figure 9.3: Sparse connectivity, viewed from above

Sparse interactions



This means that even though *direct* connections in a convolutional net are very sparse, units in the deeper layers can be *indirectly* connected to all or most of the input image

Figure 9.4: The receptive field of the units in the deeper layers of a convolutional network

Parameter sharing

- Using same parameter for more than one function in a model
- Neural net, weight matrix is used once when computing the output of a layer
- In convolutional neural net, each member of the kernel is used at every position of the input (except perhaps some of the boundary pixels, depending on the design decisions regarding the boundary).
- Parameter sharing means that learning a separate set of parameters for every location, we learn only one set.
- Does not affect the runtime of forward propagation—it is still $O(k \times n)$ —but it does further reduce the storage requirements of the model to k parameters.
- “ k ” is usually several orders of magnitude less than m .
- Since m and n are usually roughly the same size, k is practically insignificant compared to $m \times n$.
- Convolution is thus dramatically more efficient than dense matrix multiplication in terms of the memory requirements and statistical efficiency.

Parameter sharing

- Black arrows indicate the connections that use a particular parameter in two different models.
- *(Top)* The black arrows indicate uses of the central element of a 3-element kernel in a convolutional model.
- Due to parameter sharing, this single parameter is used at all input locations.
- *(Bottom)* The single black arrow indicates the use of the central element of the weight matrix in a fully connected model. This model has no parameter sharing so the parameter is used only once.

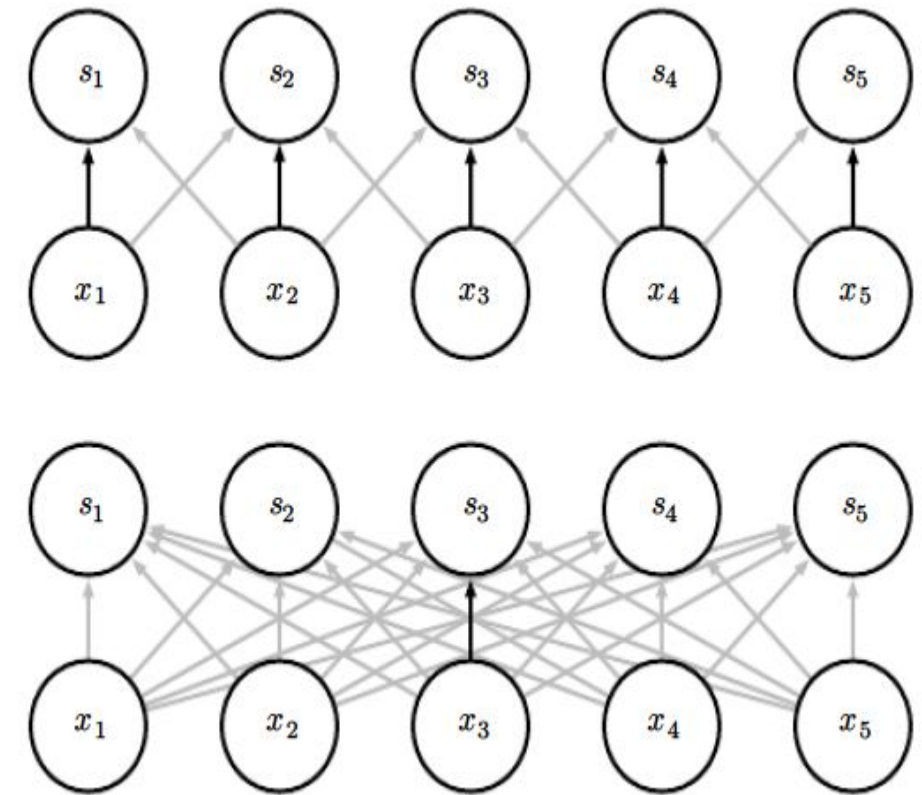
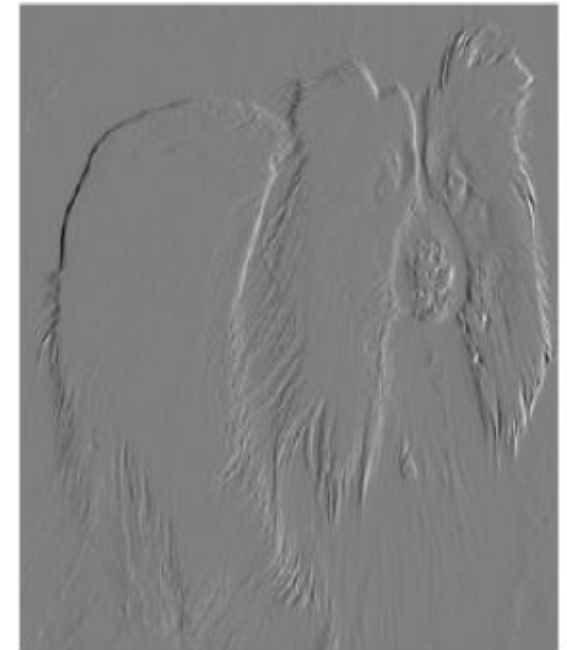


Figure 9.5: Parameter sharing:

Parameter sharing

- First two principles in action, figure shows how sparse connectivity and parameter sharing can dramatically improve the efficiency of a linear function for detecting edges in an image



□ The image on the right was formed by taking **each pixel in the original image** and **subtracting the value of its neighboring pixel on the left**.

□ Both images are 280 pixels tall, input image is 320 pixels wide while the output image is 319 pixels wide.

This transformation can be described by a convolution kernel containing two elements, and requires $319 \times 280 \times 3 = 267,960$ floating point operations (two multiplications and one addition per output pixel) to **compute using convolution**

with a matrix multiplication would take $320 \times 280 \times 319 \times 280$, or over eight billion, entries in the matrix, making convolution four billion times more efficient for representing this transformation

□ Matrix multiplication algorithm performs over sixteen billion floating point operations, making convolution roughly 60,000 times more efficient computationally.

□ Most of the matrix entries would be zero. If we stored only the nonzero entries of the matrix, then both matrix multiplication and convolution would require the same number of floating point operations to compute

□ The matrix would still need to contain $2 \times 319 \times 280 = 178,640$ entries. Convolution is an extremely efficient way of describing transformations that apply the same linear transformation of a small, local region across the entire input.

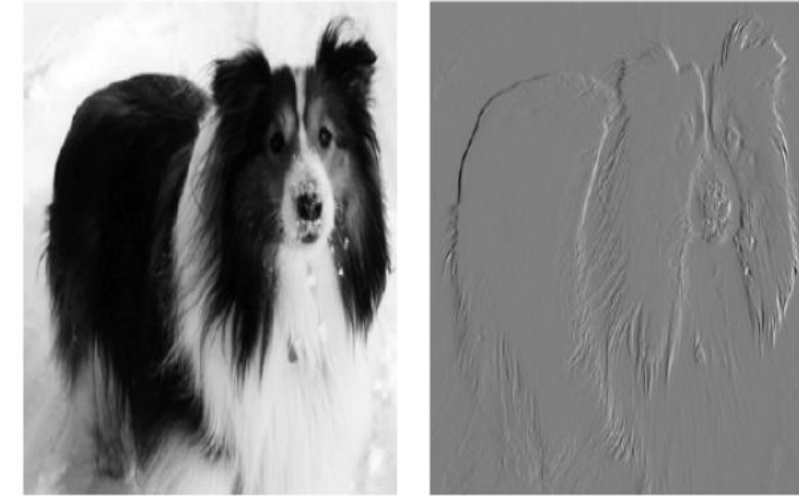


Figure 9.6: *Efficiency of edge detection.*

Convolution

- Some kinds of data cannot be processed by neural networks defined by matrix multiplication with a fixed-shape matrix.
- Convolution enables processing of some of these kinds of data.
- Convolution is not naturally equivariant to some other transformations - Scale or Rotation of an image

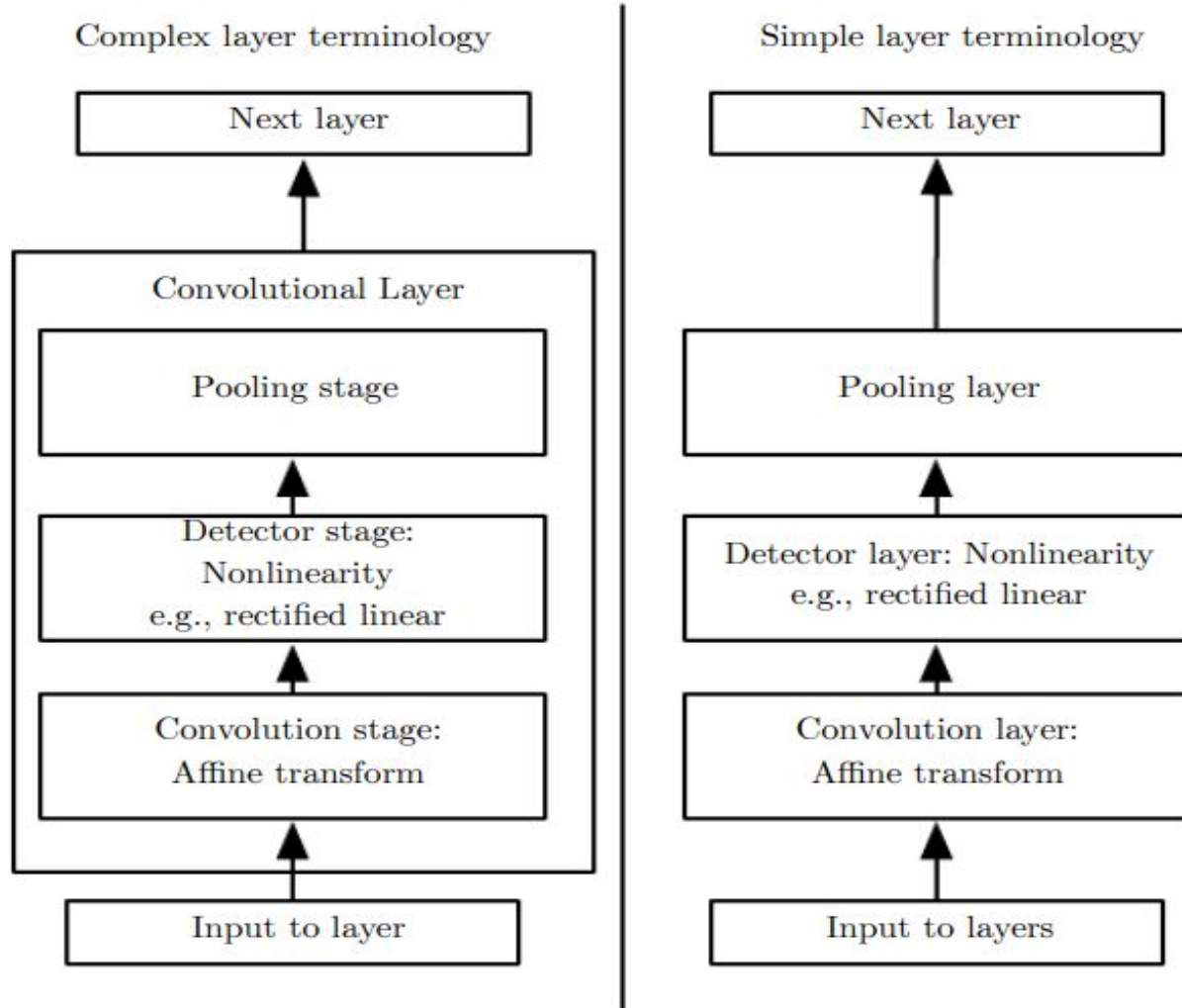
Pooling

- Convolutional network consists of three stages
 - Layer performs several **convolutions** in parallel
 - Each linear activation is run through a **nonlinear activation function**, such as the rectified linear activation function. This stage is sometimes **called the detector stage**
 - Use a **pooling** function to modify the output of the layer

Pooling -Advantages

- Feature Extraction
- Dimensionality reduction
- Reduction in computational time
- Generalization

Pooling



(Left) The convolutional net is viewed as a small number of relatively complex layers, with each layer having many “stages.”

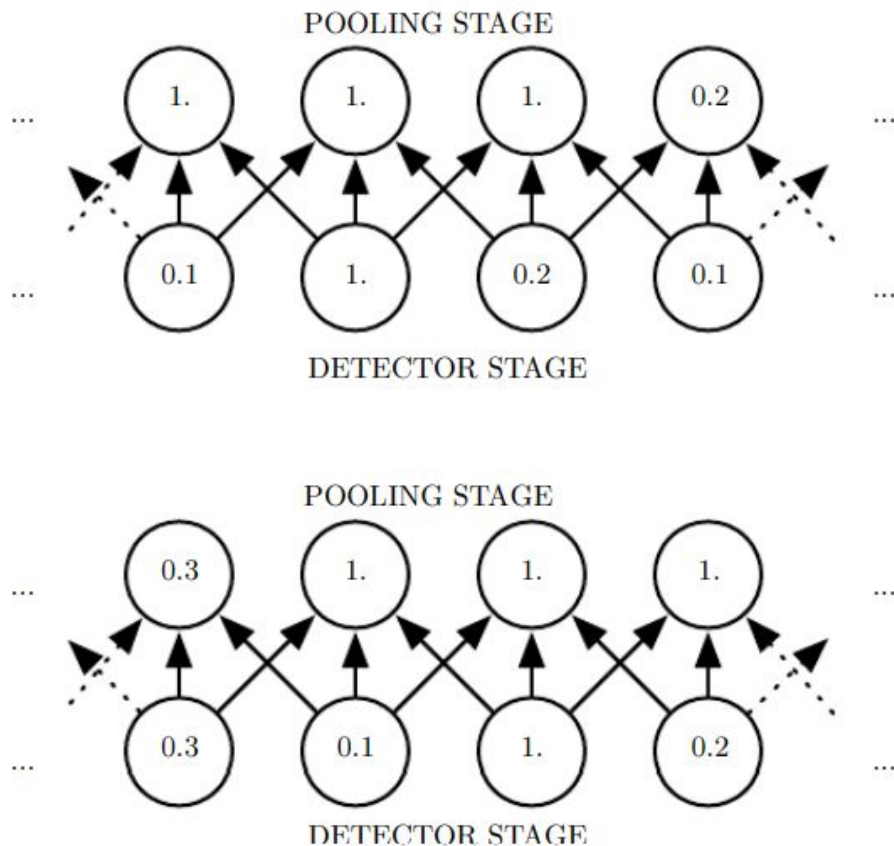
(Right) The convolutional net is viewed as a larger number of simple layers; every step of processing is regarded as a layer in its own right. This means that not every “layer” has parameters.

Figure 9.7: The components of a typical convolutional neural network layer

Pooling

- A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby outputs.
- **For example**, the max pooling operation reports the maximum output within a rectangular neighborhood
- Pooling functions include
 - ☐ **Average of a rectangular** neighborhood
 - ☐ L2 norm of a rectangular neighborhood
 - ☐ Weighted average based on the distance from the central pixel

Pooling

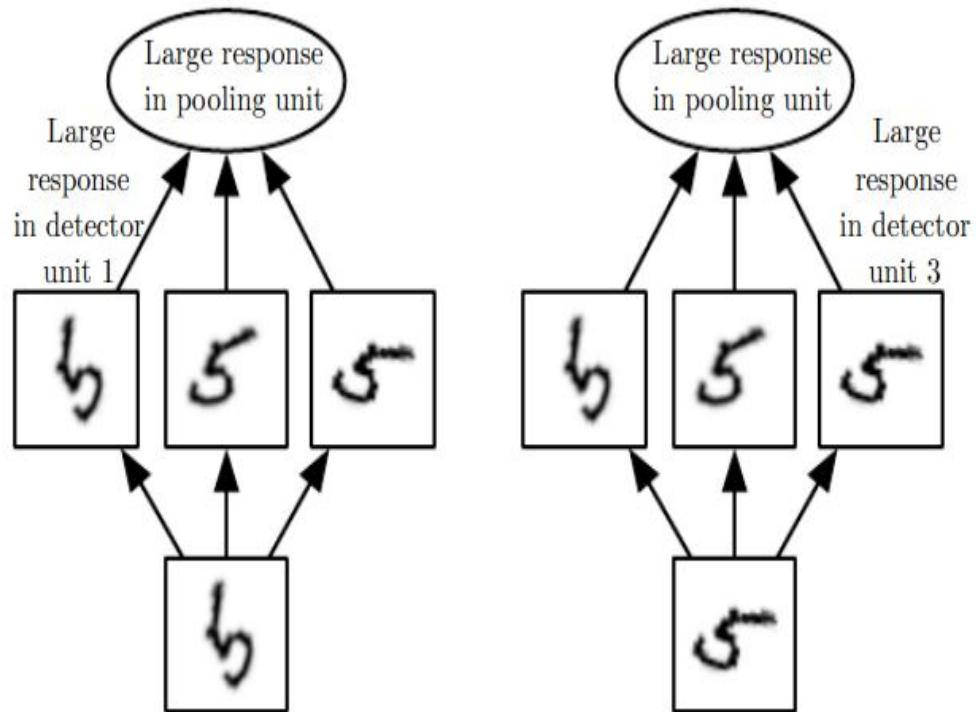


(Top) □ A view of the middle of the output of a convolutional layer. The **bottom row** shows outputs of the **nonlinearity**. The **top row** shows the outputs of **max pooling**, with a stride of one pixel between pooling regions and a pooling region **width of three** pixels.

(Bottom) □ A view of the same network, after the input has been **shifted to the right by one** pixel. Every value in the **bottom row has changed**, but only **half of the values in the top row have changed**, because the max pooling units are only **sensitive to the maximum value in the neighborhood**, not its **exact location**.

Figure 9.8: Max pooling introduces invariance

Pooling



Set of three learned filters and a max pooling unit can learn to become

- Invariant to rotation
- All three filters are intended to detect a hand-written 5
- Corresponding filter will match it and cause a large activation in a detector unit
- We show here how the network processes two different inputs, resulting in two different detector units being activated.

Figure 9.9: Example of learned invariances

Pooling

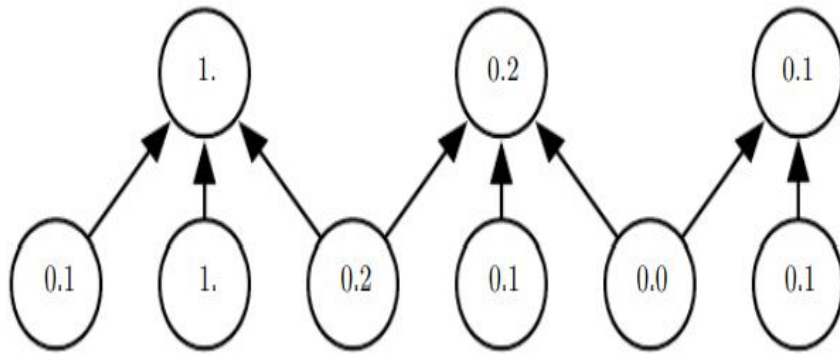
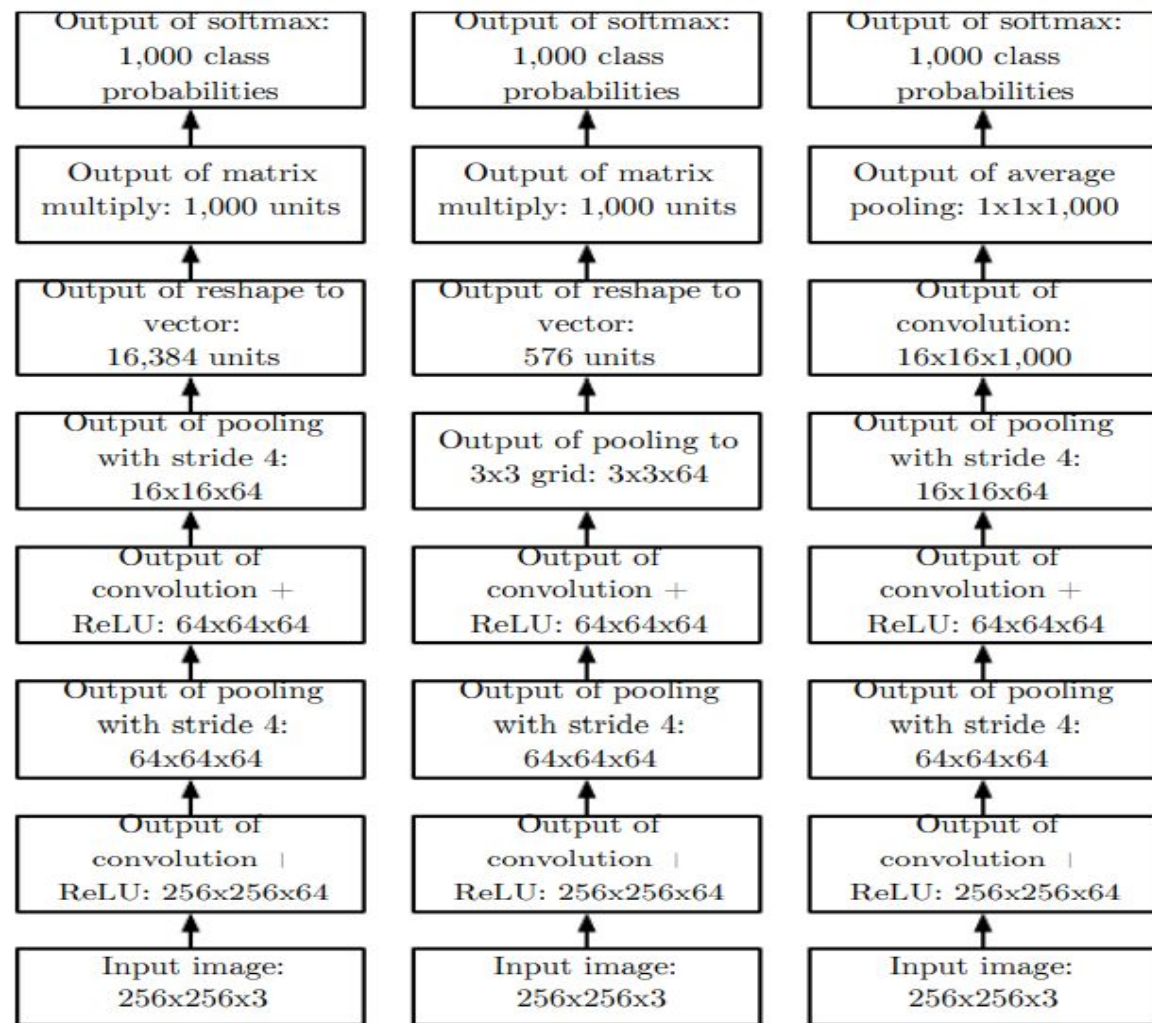


Figure 9.10: Pooling with downsampling

- Use max-pooling with a pool width of three and a stride between pools of two
- This reduces the representation size by a factor of two, which reduces the computational and statistical burden on the next layer
- Note that the rightmost pooling region has a smaller size, but must be included if we do not want to ignore some of the detector units.

Pooling



(Left) A convolutional network that processes a fixed image size. After alternating between convolution and pooling for a few layers, the tensor for the convolutional feature map is reshaped to flatten out the spatial dimensions. The rest of the network is an ordinary feedforward network classifier,

(Center) A convolutional network that processes a variable-sized image, but still maintains a fully connected section. This network uses a pooling operation with variably-sized pools but a fixed number of pools, in order to provide a fixed-size vector of 576 units to the fully connected portion of the network.

(Right) A convolutional network that does not have any fully connected weight layer. Instead, the last convolutional layer outputs one feature map per class. The model presumably learns a map of how likely each class is to occur at each spatial location. Averaging a feature map down to a single value provides the argument to the softmax classifier at the top.

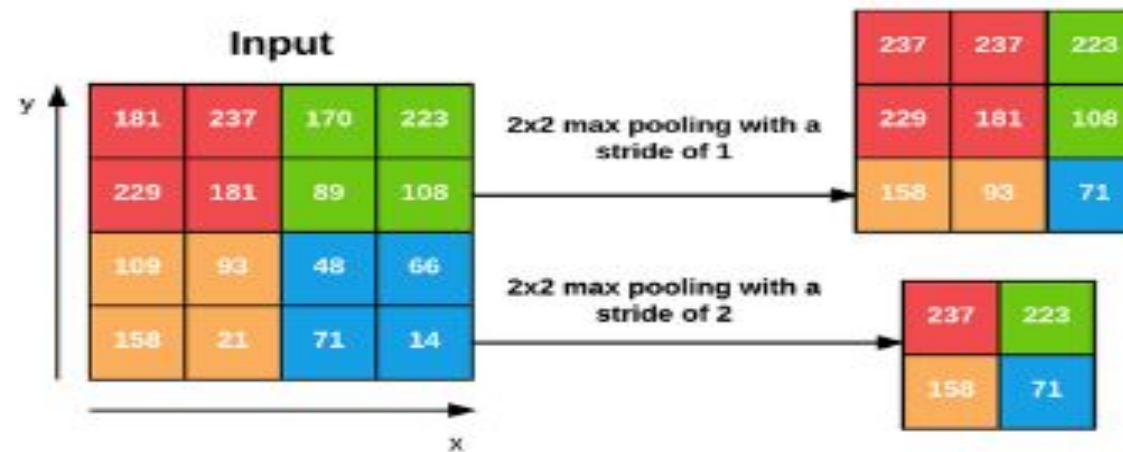
Figure 9.11: Examples of architectures for classification with convolutional networks

Image dimension

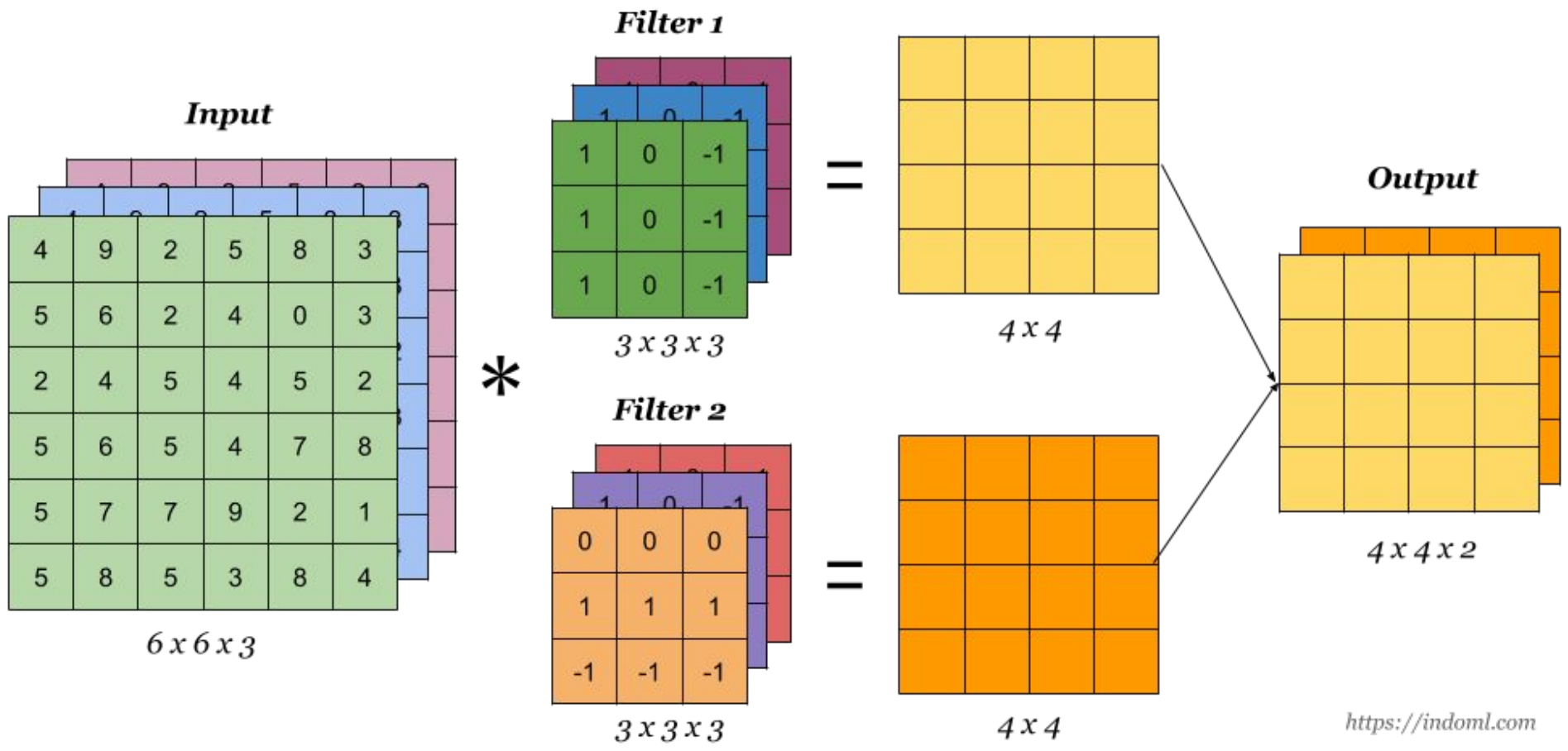
- Stride is the number of pixels shifts over the input matrix. For padding p , filter size $f*f$ and input image size $n * n$ and stride ' s ' our output image dimension will be
- $[(n + 2p - f + 1) / s + 1] * [(n + 2p - f + 1) / s + 1]$

Pooling

CONVOLUTIONAL NEURAL NETWORKS (CNNs) AND LAYER TYPES



Convolution



<https://indoml.com>

MaxPool

The most widely used pooling technique is **max pooling** since it captures the features of maximum importance with it.

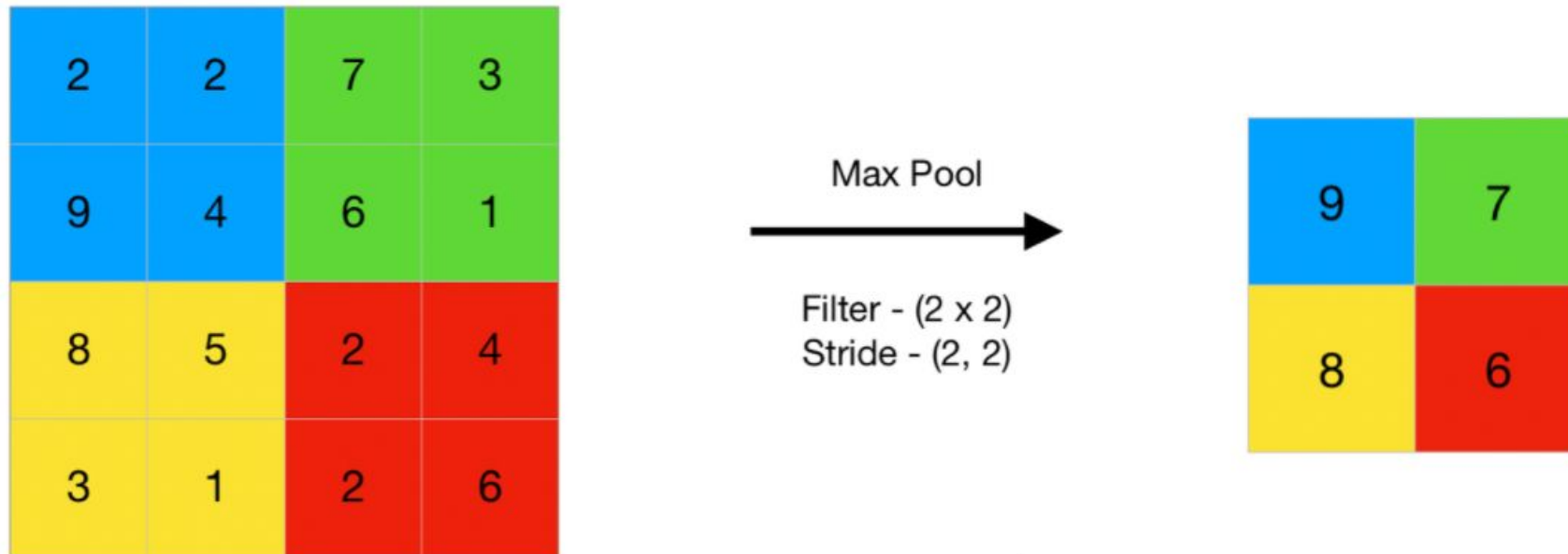
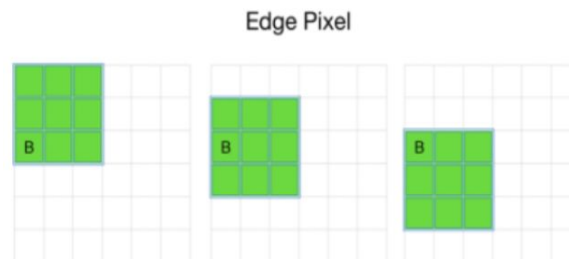
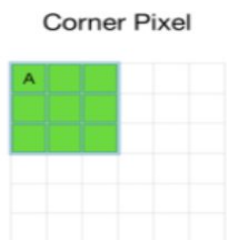


Image Source: Google Images

Problem with Simple Convolution Layers

- CNN gives results For a grayscale image of size $(n \times n)$ with $(f \times f)$ filter/kernel size is $(n - f + 1) \times (n - f + 1)$.
- Convolution operation with a (8×8) image and (3×3) filter the output image size will be (6×6) .
- When processing images, the output of the layers is shrunk in comparison to the input
- Filters do **not focus on the corners** when it moves on the pixels
- For example,

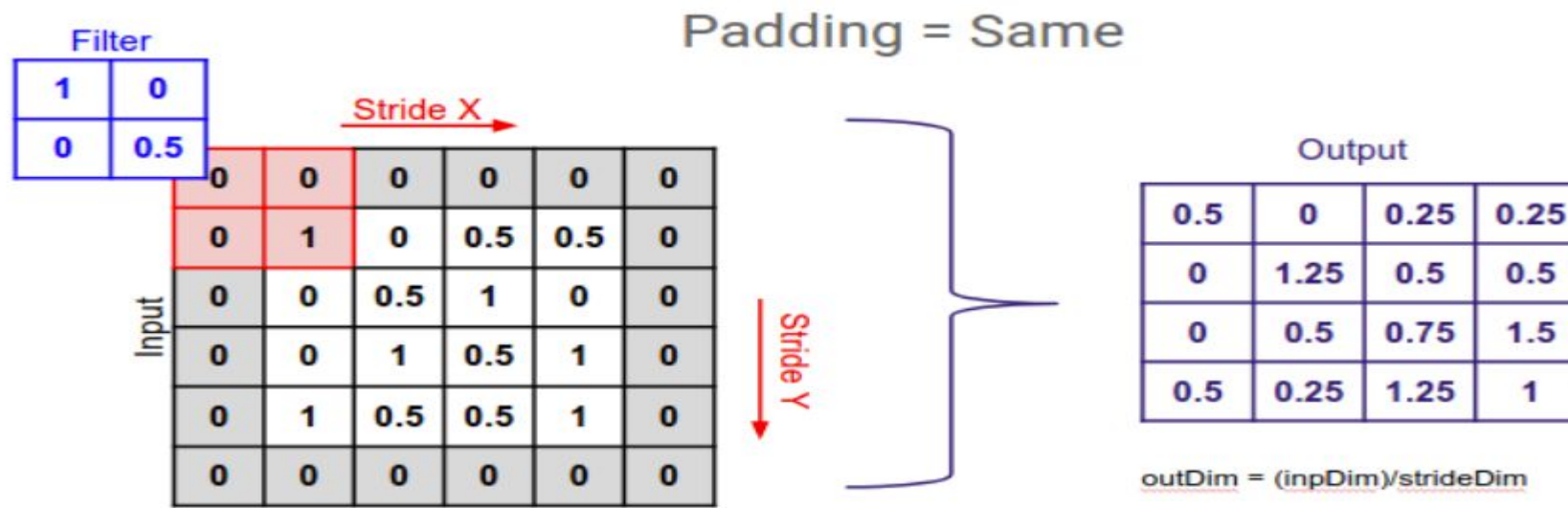


Problem with Simple Convolution Layers

- Corner pixel A is coming under the filter in only one movement where B is coming in three movements and C is coming under 9 movements.
- Model will work **with pixel C very fine** and it will **misinterpret pixel A**
- Causes **loss of information available** in the corners and also the **output from the layers is reduced**
- Reduced information will create **confusion for the next** layers
- This problem of the model can be reduced by the **padding layers**.

What is Padding?

- convolutional layers **reduce the size** of the output
- To **increase the size** of the output and **save the information** presented in the **corners** padding layers are used
- Adds extra rows and columns on the outer dimension of the images

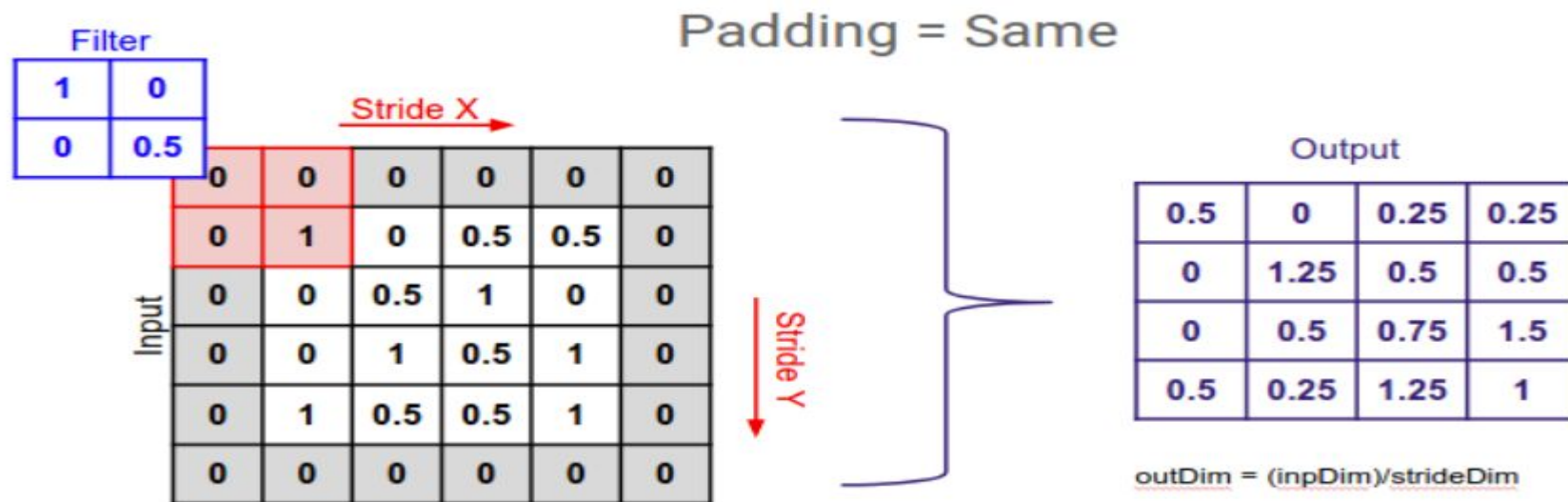


Types of Padding

- Same padding
- Causal padding
- Valid padding

Same padding

- Padding layers **append zero values** in the outer frame of the images or data
- Filter **can cover the edge** of the matrix and make the inference with them too



Without Padding

```
model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28,
28, 1)))
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
model.add(Conv2D(128, kernel_size=(3, 3), activation='relu'))
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18496
conv2d_2 (Conv2D)	(None, 22, 22, 128)	73856
Total params: 92,672		
Trainable params: 92,672		
Non-trainable params: 0		

Same padding

```
models = Sequential()
models.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=
(28, 28, 1))), padding='same'))
models.add(Conv2D(64, kernel_size=(3, 3), activation='relu',
padding='same'))
models.add(Conv2D(128, kernel_size=(3, 3), activation='relu',
padding='same'))
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 28, 28, 32)	320
conv2d_7 (Conv2D)	(None, 28, 28, 64)	18496
conv2d_8 (Conv2D)	(None, 28, 28, 128)	73856
Total params: 92,672		
Trainable params: 92,672		
Non-trainable params: 0		

Valid padding

- This type of padding can be considered as **no padding**.
- Don't apply padding but assume that every pixel of the image is valid so that the input can get fully covered by the filter wherein a **simple model assumes** corners are invalid

```
modelv = Sequential()
modelv.add(Conv2D(32, kernel_size=(3, 3), activation='relu',
input_shape=input_shape, padding = 'valid'))
modelv.add(Conv2D(64, kernel_size=(3, 3), activation='relu', padding =
'valid'))
modelv.add(Conv2D(128, kernel_size=(3, 3), activation='relu', padding =
'valid'))
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 26, 26, 32)	320
conv2d_4 (Conv2D)	(None, 24, 24, 64)	18496
conv2d_5 (Conv2D)	(None, 22, 22, 128)	73856
Total params: 92,672		
Trainable params: 92,672		
Non-trainable params: 0		

Same Vs Valid Padding

- **Same padding**, every point/pixel value while learning the model is used
 - Output size is the same as the input size (when stride=1)
- In **VALID drop** right and/or bottom cells if filter and stride don't fully cover the input image.
- Padding is called **valid** because for this padding only the **valid and original elements of the input image are considered**
 - Elements on the right and the bottom of the image tend to be ignored
 - How many elements are ignored depends on the size of the kernel and the stride

Same Vs Valid Padding

"Valid"= without padding:

```

inputs:      1  2  3  4  5  6  7  8  9  10 11 (12 13)
              |-----|
                      |-----|
                                |-----|

```

dropped

"Same"= with zero-padding:

```

           pad|                                     |pad
inputs:    0 | 1  2  3  4  5  6  7  8  9  10 11 12 13 | 0  0
              |-----|
                      |-----|
                                |-----|

```

In the above example we have:

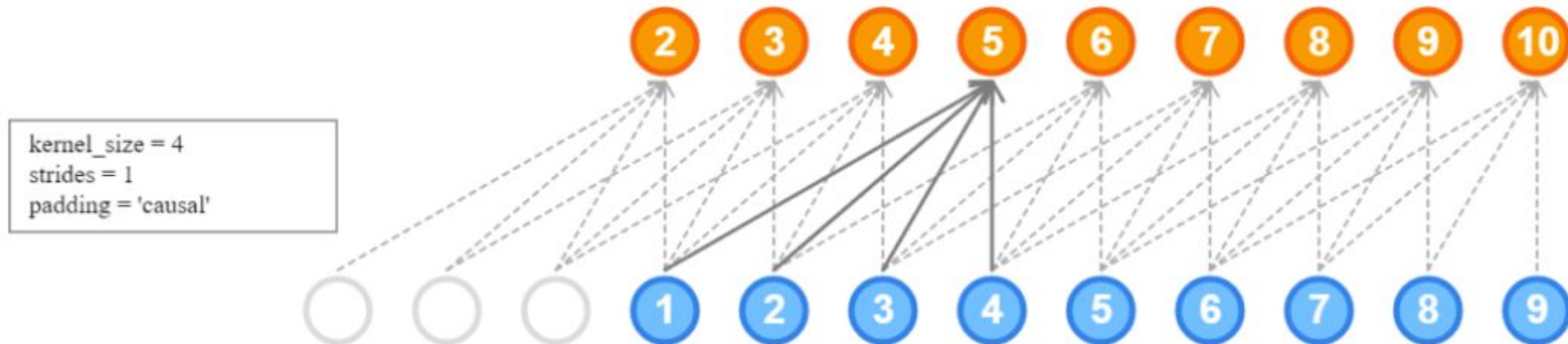
- Input width = 13
- Filter width = 6
- Stride = 5

Causal padding

- Special type of padding and basically works with the **one-dimensional convolutional layers**.
- Used majorly in **time series analysis**
- Time series is sequential data helps in **adding zeros at the start** of the data
- Helps in predicting the values of early time steps
- Every layer is using the **learnt part of the previous layer** and again the **learnt part will go to the next layer**
- If Forecasted value can be used in forecasting the next time step value then it can become more **helpful and accurate**.

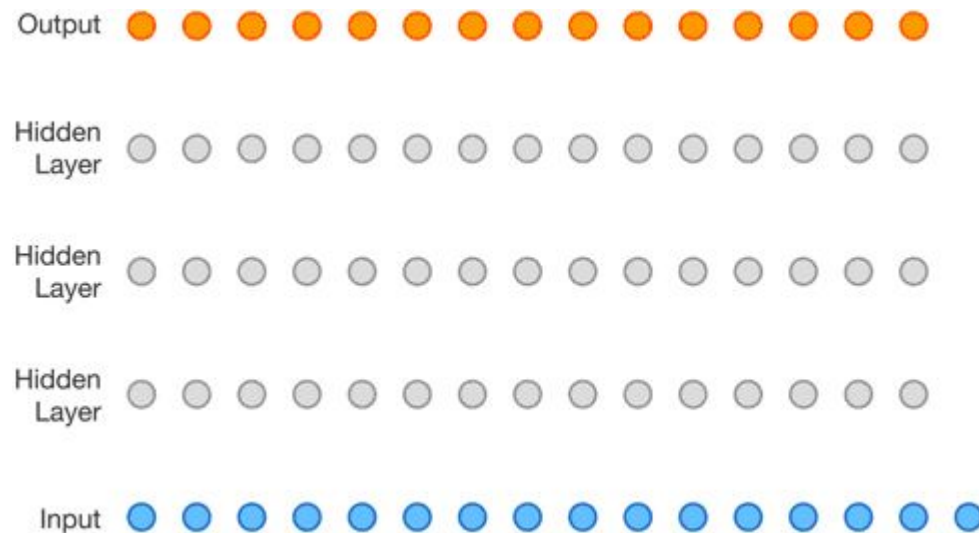
Causal padding

- A model architecture consisting of causal padding with a kernel size of four can be represented as:



Causal padding

- whole time series model architecture with a fully connected layer and causal padding can be like:



Causal padding

```
from keras.layers import Conv1D
modelc = Sequential()
modelc.add(Conv1D(32, kernel_size=4, activation='relu', input_shape=(28, 28, 1), padding='causal'))
modelc.add(Conv1D(64, kernel_size=4, activation='relu', padding='causal'))
modelc.add(Conv1D(128, kernel_size=4, activation='relu', padding='causal'))
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
=====		
conv1d_1 (Conv1D)	(None, 28, 28, 32)	160

conv1d_2 (Conv1D)	(None, 28, 28, 64)	8256

conv1d_3 (Conv1D)	(None, 28, 28, 128)	32896
=====		
Total params: 41,312		
Trainable params: 41,312		
Non-trainable params: 0		

Variants of the Basic Convolution Function

- Useful properties of the functions used in neural networks.
- **Convolution** - is an operation that consists of many applications of convolution in parallel
 - ☐ This is because convolution with a single kernel can only extract one kind of feature, at many spatial locations
 - ☐ Need each layer of our network to extract many kinds of features, at many locations
- **Input** is usually not just a grid of real values.
 - ☐ It is a grid of vector-valued observations
 - ☐ Input and output of the convolution is 3-D tensor
 - ☐ Software implementations usually work in batch mode, hence use 4-D tensors, with the fourth axis as batch

Variants of the Basic Convolution Function

- Assume a 4-D kernel tensor K with element $K_{i,j,k,l}$ giving the connection strength between a unit in channel i of the output and a unit in channel j of the input, with an offset of k rows and l columns between the output unit and the input unit.
- Assume our input consists of observed data V with element $V_{i,j,k}$ giving the value of the input unit within channel i at row j and column k . Assume our output consists of Z with the same format as V . If Z is produced by convolving K across V without flipping K , then

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$$

where the summation over l , m and n is over all values for which the tensor indexing operations inside the summation is valid. In linear algebra notation, we index into arrays using a 1 for the first entry. This necessitates the -1 in the above formula. Programming languages such as C and Python index starting from 0, rendering the above expression even simpler.

Variants of the Basic Convolution Function

- Skipping some positions of the kernel in order to reduce the computational cost
 - Downsampling
- Sampling every 's' pixels in each direction in the output, then we can define a downsampled convolution function c such that

$$Z_{i,j,k} = c(\mathbf{K}, \mathbf{V}, s)_{i,j,k} = \sum_{l,m,n} \left[V_{l,(j-1) \times s + m, (k-1) \times s + n} K_{i,l,m,n} \right]$$

We refer to 's' as the stride of this downsampled convolution.

Variants of the Basic Convolution Function

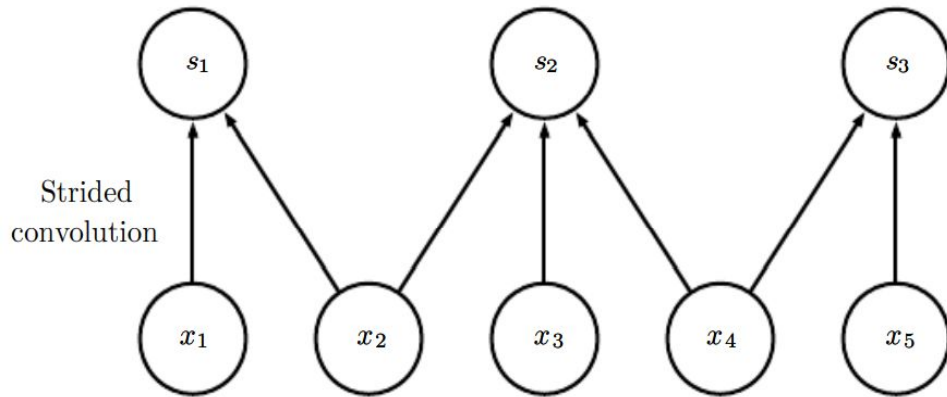
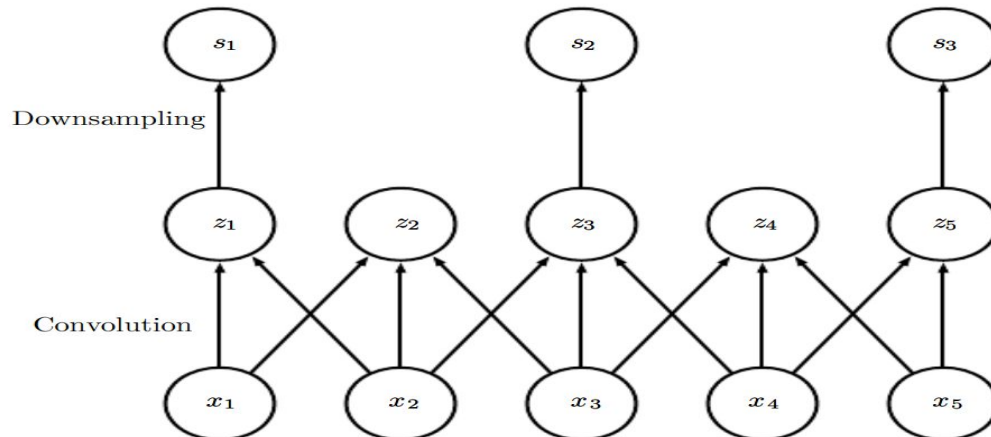


Figure 9.12: Convolution with a stride.

□ Use a stride of two.

□ (Top) Convolution with a stride length of two implemented in a single operation.



□ (Bottom) Convolution with a stride greater than one pixel is mathematically equivalent to convolution with unit stride followed by downsampling. Obviously, the two-step approach involving downsampling is computationally wasteful, because it computes many values that are then discarded.

Variants of the Basic Convolution Function

- Essential feature of any convolutional network implementation is the ability to **implicitly zero-pad** the input V in order to make it wider
- Without padding width of the representation **shrinks by one pixel** less than the kernel width at each layer
- **Zero padding** the input allows us to control the kernel width and the size of the output independently.
- Without zero padding, we are forced to choose between shrinking the spatial extent of the network rapidly and using small kernels—both scenarios that significantly limit the expressive power of the network(Fig 9.13)

Variants of the Basic Convolution Function

- Three special cases of the zero-padding
- **One** is the extreme case in which no zero-padding is used whatsoever, and the convolution kernel is only allowed to visit positions where the entire kernel is contained entirely within the image.(valid convolution)
- All pixels in output are a function of the same number of pixels in input, so the behavior of an output pixel is somewhat more regular. However, the size of the output shrinks at each layer. If the input image has width m and the kernel has width k , the output will be of width $m - k + 1$.
- Since the shrinkage is greater than 0, it limits the number of convolutional layers that can be included in the network.
- As layers are added, the spatial dimension of the network will eventually drop to 1×1 , at which point additional layers cannot meaningfully be considered convolutional.

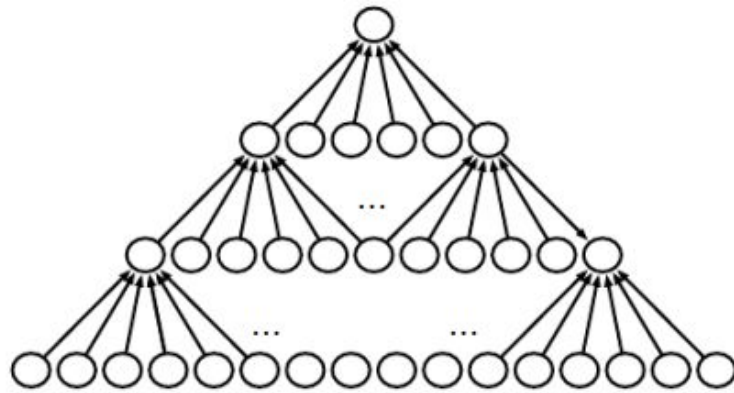
Variants of the Basic Convolution Function

- **Second** special case of the zero-padding setting is when just enough zero-padding is added to keep the size of the output equal to the size of the input (Same convolution)
- The input pixels near the border influence fewer output pixels than the input pixels near the center.
- This can make the border pixels somewhat underrepresented in the model.
- **Third** technique refers to as full convolution, in which enough zeroes are added for every pixel to be visited k times in each direction, resulting in an output image of width $m + k - 1$.
- In this case, the output pixels near the border are a function of fewer pixels than the output pixels near the center. This can make it difficult to learn a single kernel that performs well at all positions in the convolutional feature map.
- The optimal amount of zero padding (in terms of test set classification accuracy) lies somewhere between “valid” and “same” convolution.

Variants of the Basic Convolution Function

Figure 9.13: The effect of zero padding on network size

□ Consider a convolutional network with a kernel of width six at every layer. In this example, we do not use any pooling, so only the convolution operation itself shrinks the network size.



(Top) In this convolutional network, we do not use any implicit zero padding. This causes the representation to shrink by five pixels at each layer. Starting from an input of sixteen pixels, we are only able to have three convolutional layers, and the last layer does not even move the kernel, so arguably only two of the layers are truly convolutional. The rate of shrinking can be mitigated by using smaller kernels, but smaller kernels are less expressive and some shrinking is inevitable in this kind of architecture.

(Bottom) By adding five implicit zeroes to each layer, we prevent the representation from shrinking with depth. This allows us to make an arbitrarily deep convolutional network.

Variants of the Basic Convolution Function

- In some cases, instead of convolution, locally connected layers are used.
- Every connection has its own weight, specified by a 6-D tensor W .
- The indices into W are respectively: i , the output channel, j , the output row, k , the output column, l , the input channel, m , the row offset within the input, and n , the column offset within the input.

The linear part of a locally connected layer is then given by

$$Z_{i,j,k} = \sum_{l,m,n} [V_{l,j+m-1,k+n-1} w_{i,j,k,l,m,n}]$$

This is called unshared convolution, because it is a similar operation to discrete convolution with a small kernel, but without sharing parameters across locations.

Variants of the Basic Convolution Function

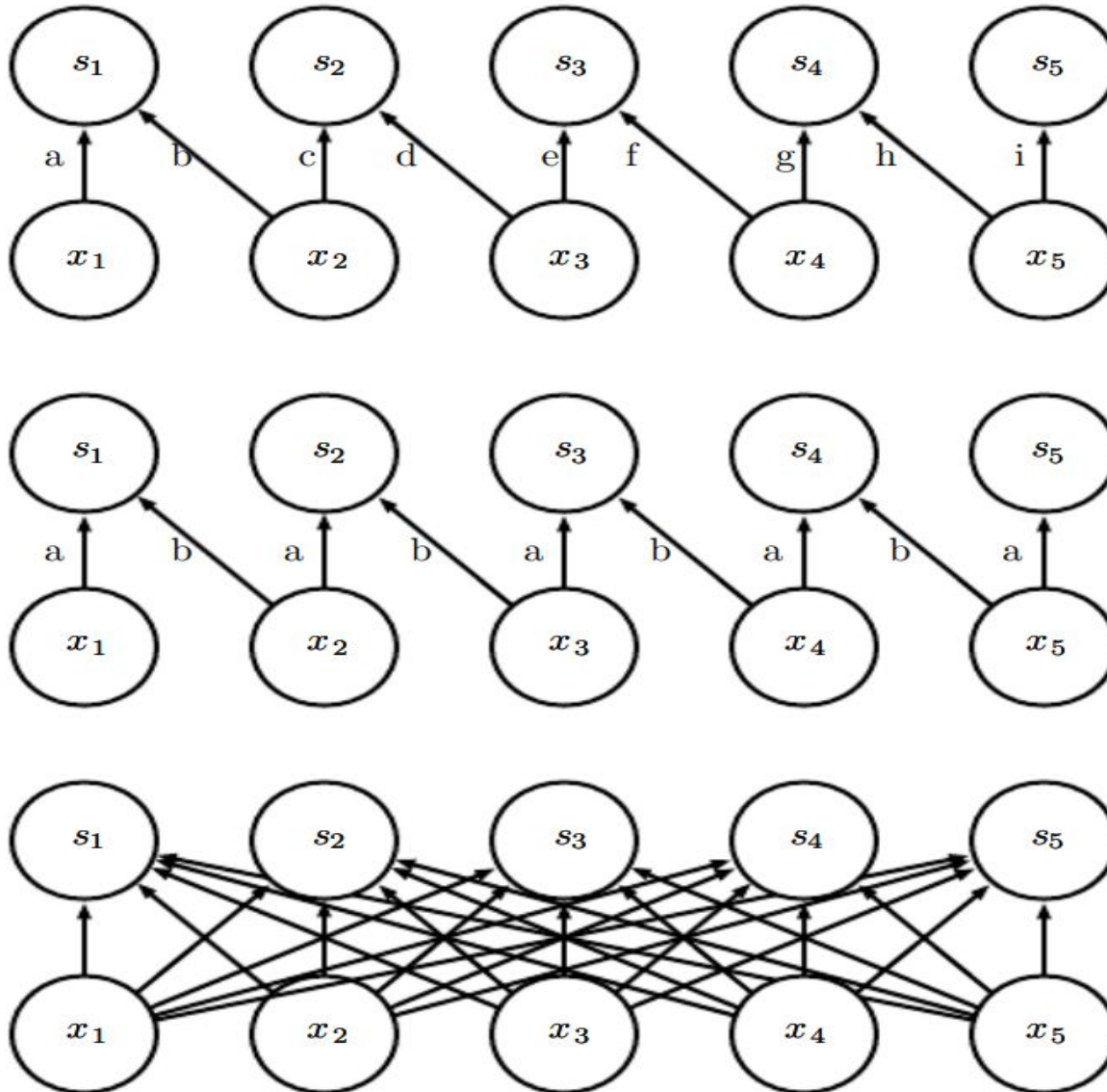


Figure 9.14: Comparison of local connections, convolution, and full connections.

(Top) A locally connected layer with a patch size of two pixels. Each edge is labeled with a unique letter to show that each edge is associated with its own weight parameter.

(Center) A convolutional layer with a kernel width of two pixels. This model has exactly the same connectivity as the locally connected layer. The difference lies not in which units interact with each other, but in how the parameters are shared. The locally connected layer has no parameter sharing. The convolutional layer uses the same two weights repeatedly across the entire input, as indicated by the repetition of the letters labeling each edge.

(Bottom) A fully connected layer resembles a locally connected layer in the sense that each edge has its own parameter (there are too many to label explicitly with letters in this diagram). However, it does not have the restricted connectivity of the locally connected layer.

Variants of the Basic Convolution Function

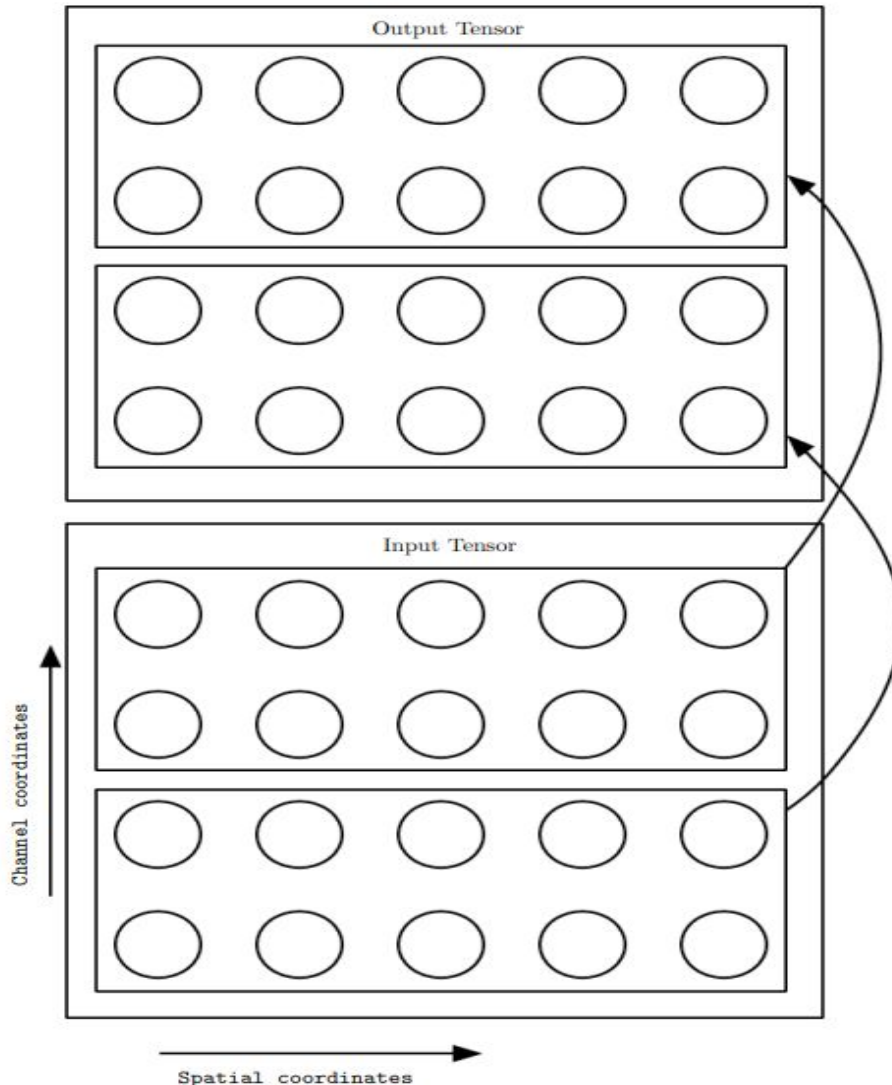


Figure 9.15: A convolutional network with the first two output channels connected to only the first two input channels, and the second two output channels connected to only the second two input channels.

Variants of the Basic Convolution Function

- Tiled convolution offers a compromise between a convolutional layer and a locally connected layer.
- Rather than learning a separate set of weights at every spatial location, a set of kernels that we rotate through as we move through space are learnt.
- Immediately neighboring locations will have different filters, like in a locally connected layer, but the memory requirements for storing the parameters will increase only by a factor of the size of this set of kernels

Variants of the Basic Convolution Function

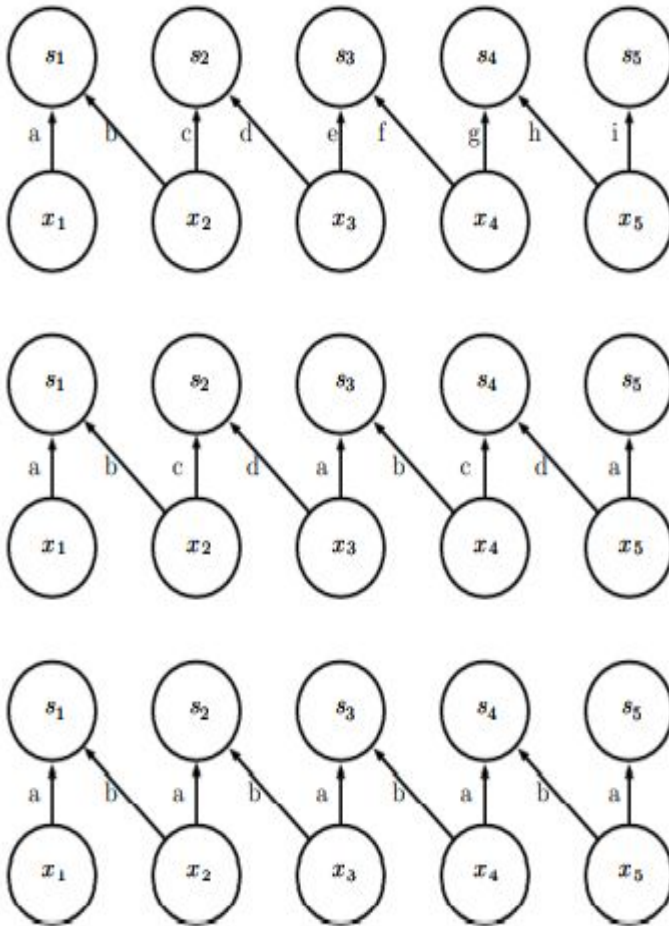
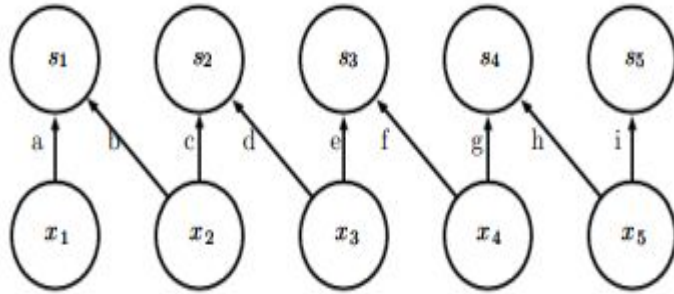


Figure 9.16: A comparison of locally connected layers, tiled convolution, and standard convolution.

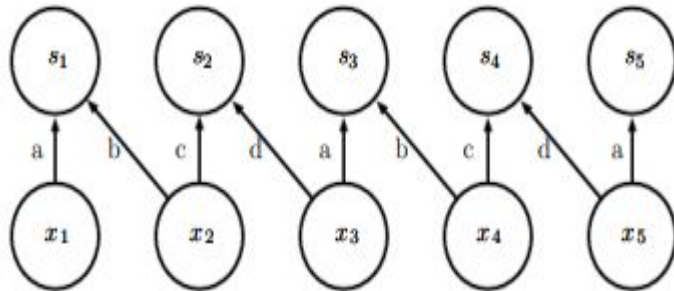
- All three have the same sets of connections between units, when the same size of kernel is used.
 - Diagram illustrates the use of a kernel that is two pixels wide.
- Differences between the methods lies in how they share parameters.

Variants of the Basic Convolution Function

Figure 9.16: A comparison of locally connected layers, tiled convolution, and standard convolution.

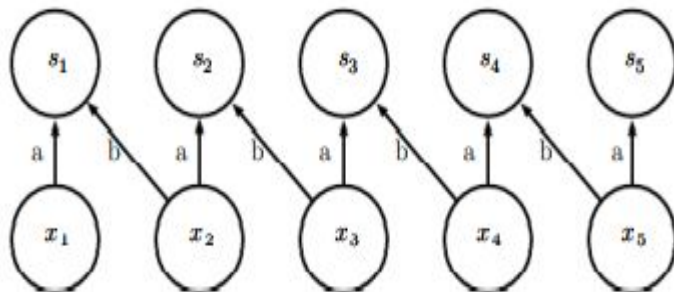


(Top) A locally connected layer has no sharing at all. Each connection has its own weight by labeling each connection with a unique letter. Tiled convolution has a set of



(Center) Tiled Convolution has set of t different kernels ($t = 2$)

□ One of these kernels has edges labeled “a” and “b,” while the other has edges labeled “c” and “d.” Each time we move one pixel to the right in the output, we move on to using a different kernel. This means that, like the locally connected layer, neighboring units in the output have different parameters. Unlike the locally connected layer, after we have gone through all t available kernels, we cycle back to the first kernel. If two output units are separated by a multiple of t steps, then they share parameters. Traditional convolution is equivalent to tiled



(Bottom) convolution with $t = 1$. There is only one kernel and it is applied everywhere, as indicated in the diagram by using the kernel with weights labeled “a” and “b” everywhere.

Variants of the Basic Convolution Function

To define tiled convolution algebraically, let k be a 6-D tensor, where two of the dimensions correspond to different locations in the output map. Rather than having a separate index for each location in the output map, output locations cycle through a set of t different choices of kernel stack in each direction. If t is equal to the output width, this is the same as a locally connected layer.

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n,j\%t+1,k\%t+1}$$

where $\%$ is the modulo operation, with $t\%t = 0$, $(t + 1)\%t = 1$, etc. It is straightforward to generalize this equation to use a different tiling range for each dimension.

Variants of the Basic Convolution Function

- Both locally connected layers and tiled convolutional layers have an interesting interaction with max-pooling
- Detector units of these layers are driven by different filters.
- If these filters learn to detect different transformed versions of the same underlying features, then the max-pooled units become invariant to the learned transformation
- Convolutional layers are hard-coded to be invariant specifically to translation

Variants of the Basic Convolution Function

- To train a convolutional network that incorporates strided convolution of kernel stack $\mathbf{K}$ applied to multi-channel image $\mathbf{V}$ with stride s as defined by $c(\mathbf{K}, \mathbf{V}, s)$ and to minimize some loss function $J(\mathbf{V}, \mathbf{K})$.
- During forward propagation, use c itself to output $\mathbf{Z}$, which is then propagated through the rest of the network and used to compute the cost function J .
- During back-propagation, we will receive a tensor $\mathbf{G}$ such

$$\mathbf{G}_{i,j,k} = \frac{\partial}{\partial \mathbf{Z}_{i,j,k}} J(\mathbf{V}, \mathbf{K})$$

Variants of the Basic Convolution Function

To train the network, we need to compute the derivatives with respect to the weights in the kernel using function

$$g(\mathbf{G}, \mathbf{V}, s)_{i,j,k,l} = \frac{\partial}{\partial K_{i,j,k,l}} J(\mathbf{V}, \mathbf{K}) = \sum_{m,n} G_{i,m,n} V_{j,(m-1) \times s + k, (n-1) \times s + l}.$$

Variants of the Basic Convolution Function

If this layer is not the bottom layer of the network, we will need to compute the gradient with respect to V in order to back-propagate the error farther down. To do so, we can use a function

$$\begin{aligned}
 h(\mathbf{K}, \mathbf{G}, s)_{i,j,k} &= \frac{\partial}{\partial V_{i,j,k}} J(\mathbf{V}, \mathbf{K}) \\
 &= \sum_{\substack{l,m \\ \text{s.t.} \\ (l-1) \times s + m = j}} \sum_{\substack{n,p \\ \text{s.t.} \\ (n-1) \times s + p = k}} \sum_q K_{q,i,m,p} G_{q,l,n}.
 \end{aligned}$$

Convolution & Pooling as an Infinitely Strong Prior

- **Prior probability distribution** illustrate about what models are reasonable, **before we have seen any data**
- **Priors** can be **weak or strong**
- A **weak prior** is a prior distribution with **high entropy**, such as a **Gaussian distribution with high variance**
 - Allows the data to move the parameters more or less freely
 - Shows that there is low confidence in **the initial value of the weight**
- A **strong prior** has very **low entropy**, such as a **Gaussian distribution with low variance**.
 - Shows a **narrow range of values about which we are confident** before learning begins

Infinitely Strong Prior

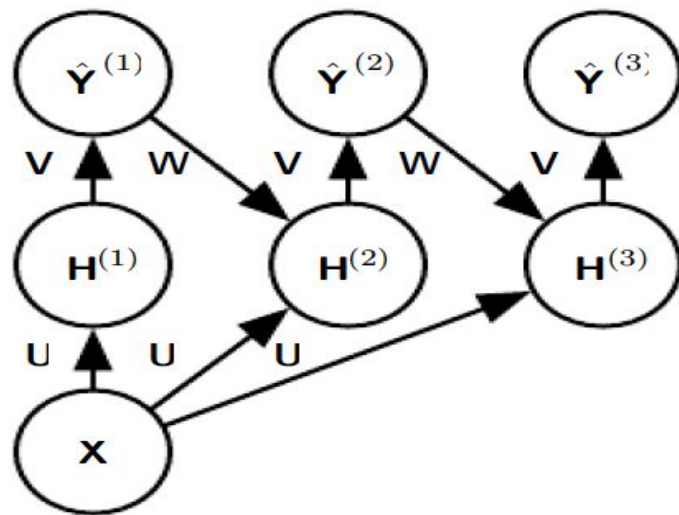
- An **infinitely strong prior** places **zero probability on some** parameters and says that these **parameter values are completely forbidden**
- Implementing a **convolutional net as a fully connected** net with an **infinitely strong prior** is computationally wasteful.
- But **convolutional net as a fully connected** net with an **infinitely strong prior** can give insights into how **convolutional nets work**.
- Convolution and pooling can cause **underfitting**
- **convolution and pooling** are only useful when the assumptions made by the prior are reasonably **accurate**
- Must **discover the concept of topology via learning**

Structured Outputs

- Convolutional networks can be used to **output a high-dimensional, structured object**, rather than just predicting a class label for a classification task or a real value for a regression task
- **Object is a tensor**, emitted by a standard convolutional layer
- **Example**, the model might emit a tensor $\mathbf{S}$, where $S_{i,j,k}$ is the probability that pixel (j, k) of the input to the network belongs to class i .

Structured Outputs

Strategy for pixel-wise labeling of images is to produce an initial guess of the image labels, then refine this initial guess using interactions between neighboring pixels



- Input is an **image tensor X** , with axes corresponding to image rows, image columns, and channels (red, green, blue).
- The goal is to output a tensor of labels $\hat{Y}$,
- This tensor has axes corresponding to image rows, image columns, and the different classes.
- Rather than outputting $\hat{Y}$ in a single shot, the recurrent network iteratively **refines its estimate $\hat{Y}$** by using a **previous estimate of $\hat{Y}$** as input for creating a new estimate
- The tensor of **convolution kernels U** is used on each step to compute the **hidden representation** given the input image.
- The kernel tensor V is used to produce an estimate of the labels **given the hidden values**.
- On all but the first step, the **kernels W** are **convolved over $\hat{Y}$** to provide input to the hidden layer

Figure 9.17: An example of a recurrent convolutional network for pixel labeling

Data types

- Data in CNN consists of several channels
- Data types with different dimensionalities and number of channels

	Single channel	Multi-channel
1-D	Audio waveform: The axis we convolve over corresponds to time. We discretize time and measure the amplitude of the waveform once per time step.	Skeleton animation data: Animations of 3-D computer-rendered characters are generated by altering the pose of a “skeleton” over time. At each point in time, the pose of the character is described by a specification of the angles of each of the joints in the character’s skeleton. Each channel in the data we feed to the convolutional model represents the angle about one axis of one joint.

Data types- Examples of different formats of data in convolutional networks.

	Single channel	Multi-channel
2-D	Audio data that has been preprocessed with a Fourier transform: We can transform the audio waveform into a 2D tensor with different rows corresponding to different frequencies and different columns corresponding to different points in time. Using convolution in the time makes the model equivariant to shifts in time. Using convolution across the frequency axis makes the model equivariant to frequency, so that the same melody played in a different octave produces the same representation but at a different height in the network's output.	Color image data: One channel contains the red pixels, one the green pixels, and one the blue pixels. The convolution kernel moves over both the horizontal and vertical axes of the image, conferring translation equivariance in both directions.

Data types

	Single channel	Multi-channel
3-D	Volumetric data: A common source of this kind of data is medical imaging technology, such as CT scans.	Color video data: One axis corresponds to time, one to the height of the video frame, and one to the width of the video frame.

Efficient Convolution Algorithms

- Modern convolutional network applications often involve networks containing **more than one million units**
- Powerful implementations **exploiting parallel computation** are essential
- Convolution is **equivalent to** converting both the input and the kernel to the **frequency domain** using a Fourier transform, performing **point-wise** multiplication of the two signals, and converting back to the **time domain** using an **inverse Fourier transform**.
- When a **d -dimensional kernel** can be expressed as the product of d vectors, one vector per dimension, the kernel is called **separable**

Efficient Convolution Algorithms

- When the **kernel is separable**, naive **convolution is inefficient**
- If the kernel is w elements wide in each dimension, then naive multidimensional convolution requires $O(wd)$ runtime and parameter storage space, while separable convolution requires $O(w \times d)$ runtime and parameter storage space.
- Devising faster ways of performing convolution or approximate convolution without harming the accuracy of the model is an active area of research.

Random or Unsupervised features

- **Expensive** part of convolutional network training is **learning the features**
- Output layer is **inexpensive due to the small number of features** provided as input to this layer after passing through several layers of pooling
- When performing **supervised training** with gradient descent, every gradient step requires a complete run of forward propagation and backward propagation through the entire network.
- One way to **reduce the cost** of convolutional network training is to use features that are **not trained in** a supervised fashion
- There are **three basic strategies** for obtaining **convolution kernels** without supervised training
 - ☐ One is to simply initialize them randomly
 - ☐ Design them by hand, for example by setting each kernel to detect edges at a certain orientation or scale.
 - ☐ Can learn the kernels with an unsupervised criterion.

Random or Unsupervised features

- For example, **apply k -means** clustering to small image patches, then use each learned **centroid as a convolution kernel**
- One can then **extract the features for the entire training set just once**, essentially constructing a new training set for the last layer.
- Learning the last layer is then typically a **linear/convex optimization** problem, like logistic regression or an SVM

Random or Unsupervised features

- **Random filters** often work surprisingly well in convolutional networks
- Provides an **inexpensive way** to choose the architecture of a convolutional network
- An intermediate approach is to learn the features, but using methods that do not require **full forward and back-propagation at** every gradient step.
- **layer-wise pretraining**, to train the first layer in isolation, then extract all features from the first layer only once, then **train the second layer** in isolation given those features, and so on.

Random or Unsupervised features

- **Unsupervised pretraining** may offer some **regularization** (prevents from overfitting) relative to supervised training
- Unsupervised pretraining allow us to train much larger architectures due to the reduced computational cost of the learning rule

Neuroscientific basis for CNN

- Part of the brain **called V1**, known as the **primary visual cortex**.
- V1 is the first area of the brain that begins to **perform significantly advanced processing of visual input**.
- Images are formed by **light arriving in the eye** and **stimulating the retina**, the **light-sensitive tissue in the back of the eye**.
- The neurons in the retina **perform simple preprocessing** of the image but do not substantially alter the way it is represented
- The image then passes through the **optic nerve and a brain region** called the **lateral geniculate nucleus**
- Carry the signal from the eye to V1, which is located at the back of the head

Neuroscientific basis for CNN

- A convolutional network layer is designed to capture three properties of V1:
 1. ☐ V1 is arranged in a **spatial map & has 2D structure** mirroring the structure of the image in the retina
 - ☐ Convolutional networks capture this property by having their features defined in terms of **two dimensional maps**.
 2. ☐ V1 contains many **simple cells** does the activity of a linear function of the image in a small, spatially localized field
 - ☐ The **detector units** of a convolutional network are designed to emulate these properties of simple cells
 3. ☐ V1 also contains many **complex cells &** these cells respond to features that are similar to those detected by simple cells, but complex cells are **invariant to small shifts in** the position of the feature.
 - ☐ This inspires the pooling units of convolutional networks.
 - ☐ Complex cells are also invariant to some **changes in lighting** that cannot be captured simply by pooling over spatial locations
 - ☐ Cross-channel pooling strategies in convolutional networks, such as maxout units

Neuroscientific basis for CNN

- As we pass through **multiple anatomical layers of the brain**, find **cells that respond to some specific concept** and are invariant to many transformations of the input
- These cells have been nicknamed “**grandmother cells**”—the idea is that a person could have a neuron that activates when seeing an image of their grandmother
- These **grandmother cells exist** in the human brain, in a region called the **medial temporal lobe**
- **Researchers tested whether individual neurons would respond to photos of famous individuals.**
- “Halle Berry neuron”: an individual neuron that is activated by the concept of Halle Berry.
- This neuron fires when a person sees a photo of Halle Berry, a drawing of Halle Berry, or even text containing the words “Halle Berry.”
- Other neurons responded to the presence of Bill Clinton, Jennifer Aniston, etc.

Neuroscientific basis for CNN

- There are many differences between convolutional networks and the mammalian vision system.

- As a brief list:

1) The human eye is mostly **very low resolution**, except for a tiny patch called the **fovea**. The fovea only observes an area about **the size of a thumbnail held** at arms length.

☐ We feel as if we can see an entire scene in high resolution, **this is an illusion** created by the **subconscious part of our brain**, as it stitches together several **glimpses of small areas**.

☐ Most **convolutional networks** actually receive **large full resolution** photographs as input.

☐ The human brain makes **several eye movements** called **saccades** to glimpse the most visually salient or task-relevant parts of a scene.

Neuroscientific basis for CNN

2) Human visual system is integrated with many **other senses**, such as hearing, and factors like our moods and thoughts.

□ **Convolutional networks so far are purely visual**

3) Human visual system does much more than just recognize objects.

Understand entire scenes including many objects and relationships between objects, and processes rich 3-D geometric information

□ **Convolutional networks have been applied to some of these problems but these applications are in their infancy.**

4) Even simple brain areas like V1 are heavily impacted by feedback from higher levels

□ **Feedback has been explored extensively in neural network models but has not yet been shown to offer a compelling improvement**

5) □ **The brain probably uses very different activation and pooling functions**

Neuroscientific basis for CNN

- In a deep, nonlinear network, it can be difficult to understand the **function of individual cells**
- Simple cells in the first layer are easier to analyze, because their responses are driven by a linear function.
- In ANN, we can display an image of the convolution kernel to see what the corresponding channel of a convolutional layer responds to
- In a **biological neural network**, we do not have access to the weights themselves.
- Put an electrode in the neuron itself, display several samples of white noise images in front of the animal's retina, and record how each of these samples causes the neuron to activate.
- Then fit a linear model to these responses in order to obtain an approximation of the neuron's weights
- This approach is known as **reverse correlation**

Neuroscientific basis for CNN

- The **Gabor function** describes the **weight at a 2-D point** in the image with 2-D coordinates, $I(x, y)$.

- The response of a simple cell to an image is given by

$$s(I) = \sum_{x \in \mathbb{X}} \sum_{y \in \mathbb{Y}} w(x, y) I(x, y).$$

$\alpha, \beta_x, \beta_y, f, \phi, x_0, y_0$,
and τ are parameters that
control the properties
of the Gabor function.

Specifically, $w(x, y)$ takes the form of a Gabor function:

$$w(x, y; \alpha, \beta_x, \beta_y, f, \phi, x_0, y_0, \tau) = \alpha \exp(-\beta_x x'^2 - \beta_y y'^2) \cos(fx' + \phi),$$

where

$$x' = (x - x_0) \cos(\tau) + (y - y_0) \sin(\tau)$$

and

$$y' = -(x - x_0) \sin(\tau) + (y - y_0) \cos(\tau).$$

Neuroscientific basis for CNN

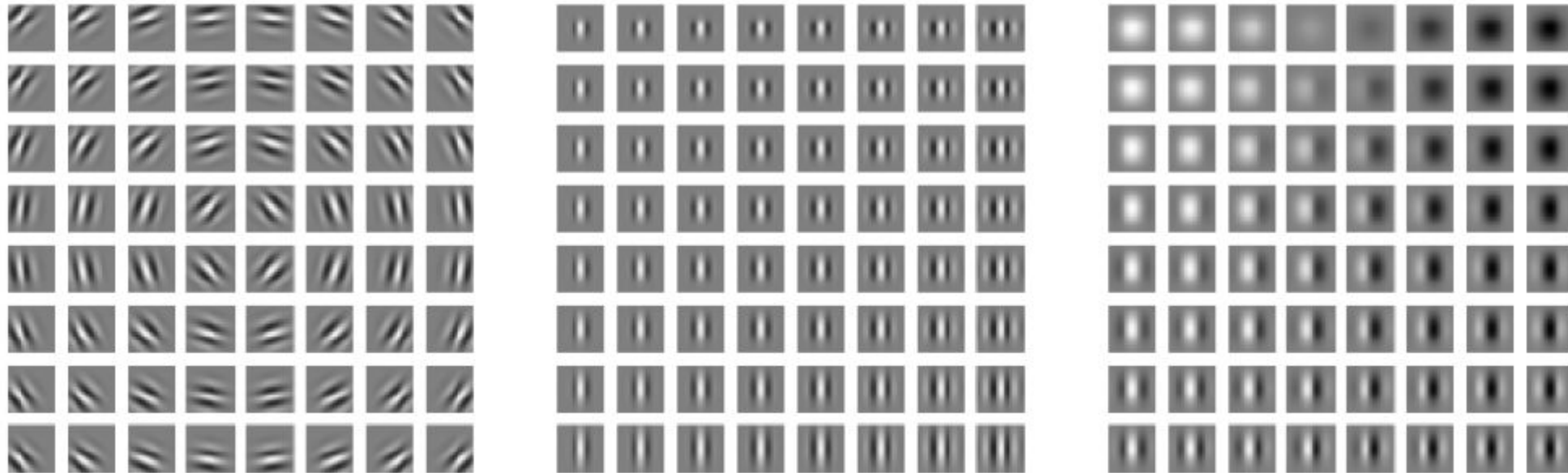
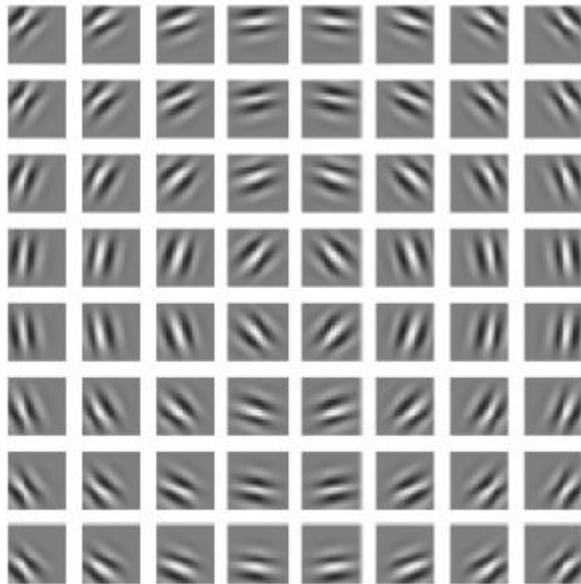


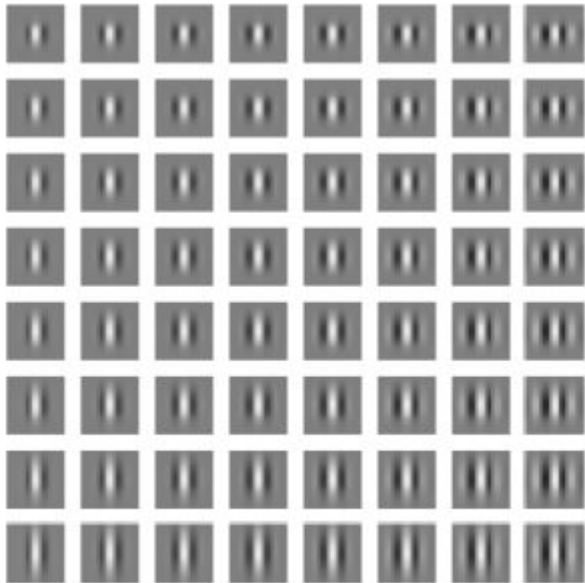
Figure 9.18: Gabor functions with a variety of parameter settings. **White** indicates large positive weight, **black** indicates large negative weight, and the background **gray** corresponds to zero weight.

Neuroscientific basis for CNN



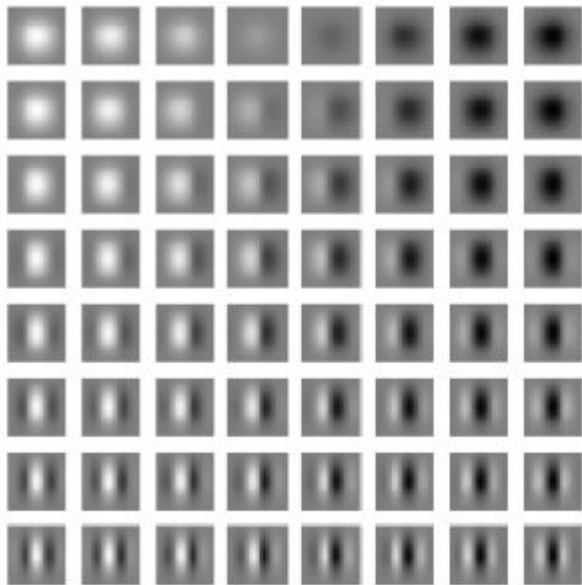
- Gabor functions with different values of the parameters that control the coordinate system: x_0 , y_0 , and τ .
- Each Gabor function in this grid is assigned a value of x_0 and y_0 proportional to its position in its grid, and τ is chosen so that each Gabor filter is sensitive to the direction radiating out from the center of the grid.
- For the other two plots, x_0 , y_0 , and τ are fixed to zero.

Neuroscientific basis for CNN



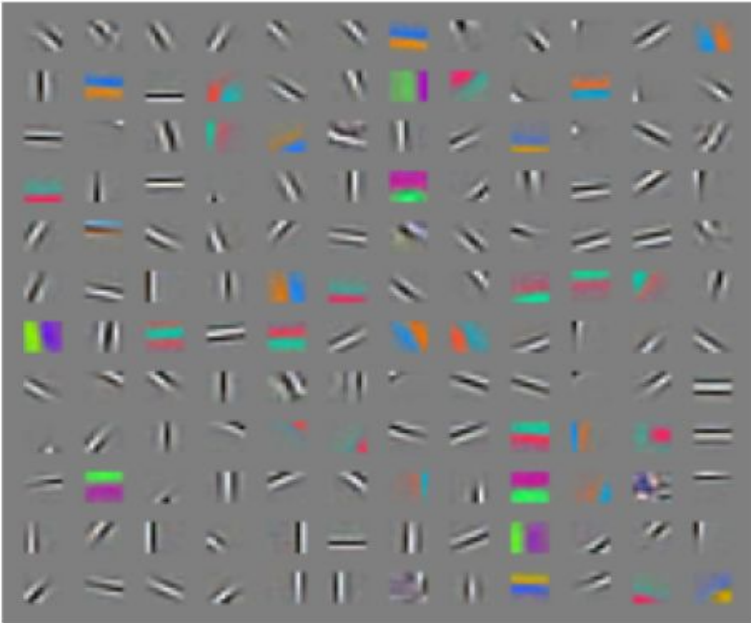
- Gabor functions with different Gaussian scale parameters β_x and β_y .
- Gabor functions are arranged in **increasing width** (decreasing β_x) as we move left to right through the grid, and increasing height (decreasing β_y) as we move top to bottom.
- For the other two plots, the β values are fixed to $1.5\times$ the image width.

Neuroscientific basis for CNN



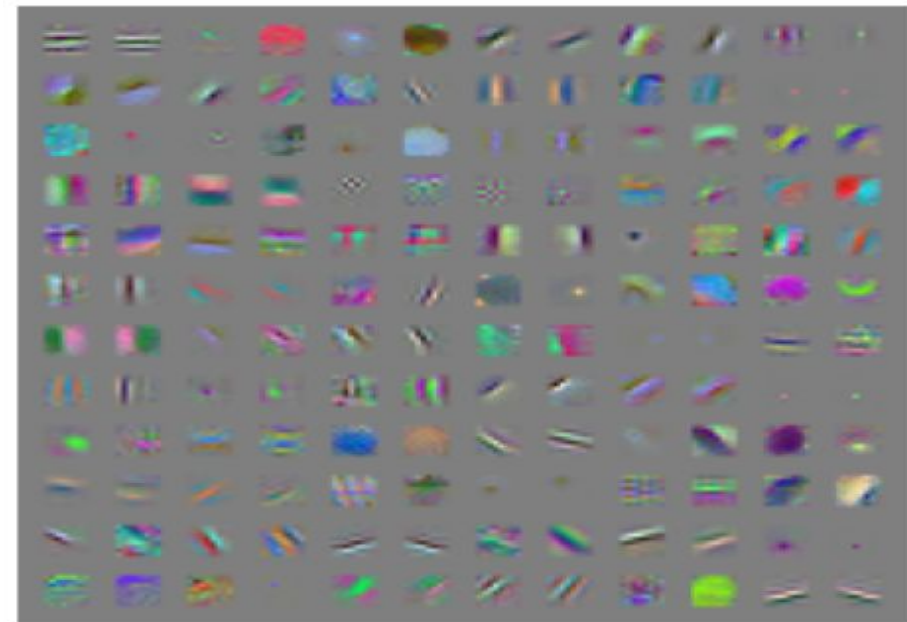
- Gabor functions with different sinusoid parameters f and φ .
- As we move top to bottom, f increases, and as we move left to right, φ increases.
- For the other two plots, φ is fixed to 0 and f is fixed to $5\times$ the image width.

Neuroscientific basis for CNN



- Many machine learning algorithms learn features that detect edges or specific colors of edges when applied to natural images.
- These feature dependent on the Gabor functions used
- (Left) Weights learned by an unsupervised learning algorithm applied to small image patches.

- (Right) Convolution kernels learned by the first layer of a fully supervised convolutional maxout network.



Thank You